

The Generator–Verifier Gap Predicts When Multi-Agent LLM Architectures Beat a Compute-Matched Single Agent

Madhavam Pratap Shahi, Kavish Soningra, and Nagendra Chaudhary

Abstract—Multi-agent architectures built from large language models are widely deployed, but reports of their benefit conflict, and practitioners have no way to anticipate whether one will help before building it. We argue two fixable problems explain the disagreement: baselines are rarely matched on compute, and no measurable task property has been identified that governs the outcome. We show that a k -agent ensemble’s accuracy decomposes exactly into *coverage*—the chance any of its candidates is correct—and *selection efficiency*, the fraction of that headroom its selector converts. We then give a latent-score model that turns a cheap pairwise probe, the generator–verifier gap G , into a parameter-free prediction of selection efficiency. Across 540 tasks in 6 families spanning the verification spectrum, 2 model scales and 8 architectures, with exact token accounting over 21,847 generations, we find that against a single agent priced at the same token budget, 47% of the apparent multi-agent gains disappear and 56% of all configurations are net negative. Diagnostics measured on 30 calibration tasks predict held-out lift with $R^2 = 0.84$ (Pearson $r = 0.92$), with no fitted parameters and without the multi-agent system ever being constructed. A case study on citation generation shows the same split in a deployed setting: an external registry check attains selection efficiency 1.00 at zero token cost, while the same model used as its own critic attains 0.00. The pattern that survives is narrow: ensembles selected by an external program verifier gain reliably, self-consistency gains where the answer distribution is peaked on the truth, and ensembles selected by the model judging itself gain nowhere. Multi-agent structure is not a general accuracy technique; it is a way of spending compute that pays off exactly when checking an answer is easier than producing one.

Index Terms—large language models, multi-agent systems, test-time compute, verification, ensemble selection, benchmarking

I. INTRODUCTION

Multi-agent architectures built from large language models are now routine engineering practice. Frameworks such as AutoGen, MetaGPT and ChatDev [1–3] let a practitioner decompose a task across several model instances that propose, critique and aggregate one another’s work, and reported gains over a single model are common. Yet the picture from the literature is contradictory. Multi-agent debate improves factuality and reasoning on some benchmarks [4]; simply sampling more answers and voting captures much of the same benefit [5, 6]; and a recent study of deployed multi-agent systems finds failure rates high enough to question whether the additional structure is doing useful work at all [7]. Practitioners deciding whether to build a multi-agent system

therefore have no principled way to anticipate the answer, and the prevailing method—build it, then measure—is expensive and often inconclusive.

We argue that two problems explain most of this disagreement, and that both are fixable.

The baseline is usually not compute-matched. A system of k agents spends roughly k times the tokens of a single agent. Comparing it against one agent that produced one answer credits the architecture for compute that any method could have spent. The honest comparison prices the single agent at the same budget—letting it think longer, or revise its own answer—and asks whether *coordination*, rather than *compute*, produced the gain. This concern has been raised for agent evaluation generally [8] and for repeated sampling in particular [9–11], but compute-matched comparison is still not standard practice in the multi-agent literature.

There is no measurable quantity that says when to expect a gain. Existing accounts are largely taxonomic: they describe what multi-agent systems do, or catalogue how they fail, without predicting where gains will appear. What is missing is a property of the *task and model together* that can be measured cheaply, in advance, and that determines the answer.

This paper supplies that quantity. Our starting point is an observation that is almost trivial once stated but that organises the entire empirical picture: an ensemble of k agents can only ever be as good as the best candidate it produces, and it realises that potential only to the extent that its selection mechanism can tell good candidates from bad. Writing p_1 for single-sample accuracy, $C(k)$ for the probability that at least one of k samples is correct, and $A_\sigma(k)$ for the accuracy of a system using selector σ ,

$$A_\sigma(k) = p_1 + \eta_\sigma(k) (C(k) - p_1), \quad (1)$$

where $\eta_\sigma(k) \in [0, 1]$ is the fraction of the available *coverage headroom* $C(k) - p_1$ that the selector converts into accuracy. Multi-agent gains require both factors to be non-trivial: headroom without a discriminating selector is wasted, and a perfect selector on a pool with no headroom has nothing to find.

The useful consequence is that both factors can be estimated *before* building the system, from a small labelled calibration set. Coverage headroom needs only repeated sampling. For selection we introduce the **generator–verifier gap** G : the probability, in excess of chance, that the model prefers a correct candidate over an incorrect one when shown both. Because G is a *pairwise* quantity while selection is *listwise*, we supply the link with a latent-score model (Section III)

that turns a measured G into a parameter-free prediction of η . Verifier-free aggregation such as self-consistency needs no verifier at all and is governed by a second diagnostic, **plurality alignment** A , which measures how well the modal answer tracks the truth.

We test this on 540 tasks spanning six families chosen to span the verification spectrum—from constraint-satisfaction problems whose answers can be checked in a line of code, to transitive-ordering puzzles where confirming an answer means redoing the derivation—across 2 model scales and 8 architectures, all with exact token accounting. Our contributions:

- 1) **A decomposition and a theory.** Equation (1) separates a multi-agent system’s accuracy into coverage and selection efficiency, and a latent-score model predicts selection efficiency from the pairwise gap G with no free parameters (Section III).
- 2) **Token-matched evidence that most multi-agent gains are compute, not coordination.** Against a single agent priced at the same token budget, 47% of the architecture-task cells that appear to gain lose that gain, and the median configuration moves from +0.036 to -0.028—a change of sign (Section V).
- 3) **A prospective diagnostic.** Measured on 30 calibration tasks and passed through the model with no fitted parameters, the diagnostics predict held-out lift with $R^2 = 0.84$ without the multi-agent system ever being built. As a binary build/don’t-build rule they are weaker—73% correct against a 61.5% majority baseline—and we report that gap rather than the raw accuracy.
- 4) **A case study in a deployed setting.** On citation generation, where fabricated output is a known failure mode, an external registry check converts nearly all available headroom at zero token cost while the same model used as its own critic performs near chance (Section VI).

Our headline practical finding is a negative result with a constructive remedy. Multi-agent structure is not a general-purpose accuracy technique; it is a way of spending compute that pays off precisely when an asymmetry exists between producing an answer and checking one. Where that asymmetry is absent, the same tokens are better spent on a single agent—and where it is present, the effective move is usually not another language-model critic but an external check, which in our experiments costs no tokens at all.

II. RELATED WORK

a) Repeated sampling and test-time compute.: Drawing many samples and keeping the best is the oldest form of test-time scaling. Chen et al. [12] introduced the unbiased $\text{pass}@k$ estimator we use for coverage; Brown et al. [9] showed coverage continues to grow over orders of magnitude of samples, and Snell et al. [10] studied how best to allocate a fixed test-time budget. This line establishes that the *coverage* factor in Equation (1) is typically large. It also makes clear that coverage alone is not accuracy: converting it requires a selector, and Stroebel et al. [11] show formally that an imperfect verifier caps the returns from resampling. Our contribution

relative to that work is to make the selector’s contribution a *measurable, predictable* quantity, tied to a diagnostic that can be run before the system exists, and to evaluate it against a compute-matched single agent rather than against a single sample.

b) Aggregation without a verifier.: Self-consistency [5] takes a majority vote over sampled reasoning chains and needs no verifier whatsoever. Li et al. [6] find that this simple sampling-and-voting procedure scales with the number of agents and often matches more elaborate multi-agent designs. Our plurality alignment diagnostic is the property these methods implicitly rely on: that the sampling distribution is peaked on the correct answer. We treat it as a second, separate mechanism rather than folding it into verification, because our data show families where voting helps while the model’s verifier is at or below chance.

c) Verifiers and judges.: Training a separate verifier is highly effective when supervision is available [13–15]. In deployed multi-agent systems the verifier is more often the same model prompted to judge, a practice whose biases—including position bias—are documented by Zheng et al. [16]. A related body of work finds that models are poor at correcting their own reasoning without external signal [17, 18], tempering the optimism of self-refinement approaches [19]. The generator-verifier gap makes this a quantity rather than a verdict: the same model can be an excellent verifier on one family and a below-chance one on another, and we measure which.

d) Multi-agent frameworks and their failures.: Debate [4] and role-structured frameworks [1–3] are the dominant designs. Cemri et al. [7] analyse why such systems fail and produce a taxonomy of failure modes. Our account is complementary and predictive rather than diagnostic: instead of classifying failures after the fact, we identify a measurable precondition for success and show it forecasts the outcome.

e) Cost-aware evaluation.: Kapoor et al. [8] argue that agent evaluations systematically neglect cost, and that controlling for it changes conclusions. We adopt that stance for multi-agent architecture specifically, reporting every result against a single agent priced at the identical token budget, and we find the correction is not cosmetic: it flips the sign of a substantial share of apparent gains.

III. FRAMEWORK

A. Setup

Fix a task distribution $\mathcal{T}$ and a generator π (a language model with a decoding temperature). For a task t , drawing a sample $y \sim \pi(t)$ yields a correct answer with probability $p_1(t)$, and we write $p_1 = \mathbb{E}_t[p_1(t)]$ for the single-sample accuracy. A k -agent ensemble draws $y_1, \dots, y_k \sim \pi(t)$ independently and applies a *selector* σ that returns one index. We write $A_\sigma(k)$ for the resulting accuracy.

Two reference selectors bound the rest. The *oracle* selector returns a correct candidate whenever one exists, so its accuracy is the coverage

$$C(k) = \mathbb{E}_t[\Pr(\exists i \leq k : y_i \text{ correct})], \quad (2)$$

which we estimate with the unbiased $\text{pass}@k$ estimator of Chen et al. [12] from a pool of $n \geq k$ samples. The *uninforma-*

tive selector chooses an index independently of correctness; its accuracy is p_1 for every k , since each candidate is marginally correct with probability p_1 .

B. The coverage–selection decomposition

Definition 1 (Selection efficiency). For a selector σ with $C(k) \neq p_1$,

$$\eta_\sigma(k) = \frac{A_\sigma(k) - p_1}{C(k) - p_1}. \quad (3)$$

Proposition 1 (Decomposition). For any selector, $A_\sigma(k) = p_1 + \eta_\sigma(k)(C(k) - p_1)$. An uninformative selector has $\eta_\sigma(k) = 0$ and the oracle has $\eta_\sigma(k) = 1$. Every selector satisfies $\eta_\sigma(k) \leq 1$; $\eta_\sigma(k) < 0$ is attainable and indicates a selector anti-correlated with correctness.

Proof. The identity is Equation (3) rearranged. For the uninformative selector, selection is independent of correctness, so the chosen candidate is correct with its marginal probability p_1 , giving $\eta = 0$. For the oracle $A_\sigma(k) = C(k)$ gives $\eta = 1$. No selector can exceed $C(k)$, since it returns one of the generated candidates and is correct only if that candidate is; hence $\eta_\sigma(k) \leq 1$. Nothing prevents $A_\sigma(k) < p_1$ —a selector that systematically prefers confident errors—so $\eta_\sigma(k)$ is not bounded below by zero. $\square$

Proposition 1 is elementary, and deliberately so: its value is that it names the two factors that any multi-agent gain must pass through, and both are separately measurable. It also makes the failure modes explicit. A selector cannot rescue a pool with no headroom ($C(k) \approx p_1$), and headroom is worthless without a selector that can find it ($\eta \approx 0$). The possibility $\eta < 0$ is not hypothetical; we observe it (Section V).

C. From pairwise discrimination to listwise selection

The decomposition is useless prospectively unless η can be estimated without building the system. We estimate it from a cheaper probe.

Definition 2 (Generator–verifier gap). Present the model with a task and two candidate answers, one correct and one incorrect, in randomised order, and ask which is correct. Let q be the probability it answers correctly. The generator–verifier gap is $G = 2q - 1 \in [-1, 1]$.

$G = 0$ means the model cannot distinguish its own correct answers from its incorrect ones; $G = 1$ means it always can; $G < 0$ means it is systematically misled. Crucially G is a pairwise quantity, while a k -agent ensemble makes a listwise choice. We bridge the two with a minimal latent-score model.

Definition 3 (Latent-score selector). The verifier assigns candidate i a score s_i , drawn independently from $\mathcal{N}(\delta, 1)$ if y_i is correct and $\mathcal{N}(0, 1)$ otherwise, and selects $\arg \max_i s_i$.

Proposition 2 (Identification and prediction). Under Definition 3, the pairwise preference probability is $q = \Phi(\delta/\sqrt{2})$, so an observed gap identifies the separation

$$\delta = \sqrt{2} \Phi^{-1}\left(\frac{1+G}{2}\right) \quad (4)$$

with no free parameters. If c of the k candidates are correct, the probability of selecting a correct one is

$$P(c, k, \delta) = \Pr\left(\max_{j \leq c} X_j > \max_{j \leq k-c} Z_j\right), \quad X_j \sim \mathcal{N}(\delta, 1), \quad Z_j \sim \mathcal{N}(0, 1), \quad (5)$$

and the predicted selector accuracy is $\hat{A}_\sigma(k) = \mathbb{E}_c[P(c, k, \delta)]$, with the expectation taken over the observed distribution of c . The predicted selection efficiency follows from Equation (3).

Proof. $s_{\text{correct}} - s_{\text{incorrect}} \sim \mathcal{N}(\delta, 2)$, so $q = \Pr(s_{\text{correct}} > s_{\text{incorrect}}) = \Phi(\delta/\sqrt{2})$, which inverts to Equation (4) since Φ is strictly increasing. Equation (5) is the definition of the argmax event conditional on c ; $P(0, k, \delta) = 0$ and $P(k, k, \delta) = 1$. Averaging over c gives $\hat{A}_\sigma(k)$. $\square$

Two things make Proposition 2 usable. First, G and the distribution of c are both estimable from a small labelled calibration set by sampling k answers per task—no multi-agent system need be built. Second, nothing is fitted: Equation (4) pins δ , and Equation (5) is evaluated by simulation. The model is intentionally the simplest that respects the pairwise/listwise distinction; it assumes scores are conditionally independent and equally dispersed, and Section V reports where those assumptions hold and where they fail.

D. Aggregation without a verifier

Self-consistency uses no verifier, so G cannot explain it. It relies instead on the sampling distribution concentrating on the truth.

Definition 4 (Plurality alignment). Let $\hat{y}$ be the modal answer among k samples. Plurality alignment is $A = \Pr(\hat{y} \text{ correct}) - p_1$, the accuracy advantage of the mode over a single sample.

A is also estimable from the same calibration pool at no extra cost, and it is the natural predictor of self-consistency’s lift. Separating G from A matters empirically: we find families where $A > 0$ while $G \leq 0$, so a system can benefit from voting while the very same model is useless as a judge.

E. Token-matched lift and the decision rule

Let $\mathcal{C}(\cdot)$ denote total tokens consumed per task, prompt plus completion. A k -agent ensemble costs roughly k times a single sample. We therefore compare it not against one sample but against the *single-agent frontier* $a(\cdot)$: the accuracy a single agent reaches when given the same budget to spend serially, by generating and then revising its own answer.

Definition 5 (Token-matched lift). For an architecture $\mathcal{A}$ with cost $\mathcal{C}(\mathcal{A})$,

$$\Lambda(\mathcal{A}) = A(\mathcal{A}) - a(\mathcal{C}(\mathcal{A})). \quad (6)$$

$\Lambda > 0$ is the claim a multi-agent architecture must support: that spending the budget across coordinating agents beats spending it inside one. Combining with Proposition 1 gives the rule we evaluate prospectively:

$$\text{build the ensemble} \iff \underbrace{\hat{\eta}_\sigma(k)(\hat{C}(k) - \hat{p}_1)}_{\text{predicted from calibration}} > \underbrace{a(\mathcal{C}(\mathcal{A})) - p_1}_{\text{sequential gain}}. \quad (7)$$

Every quantity on the left is measurable in advance from Proposition 2 and repeated sampling; the right-hand side requires only running one agent at a larger budget. Section V tests both the numerical prediction and the binary decision it implies.

IV. EXPERIMENTAL SETUP

A. Task families

We need tasks that span the verification spectrum, not a single benchmark. We use six families, 90 tasks each (540 total), chosen so that the cost of *checking* an answer varies by orders of magnitude while the cost of *producing* one does not. *CSP (synthetic)*. “Find the smallest integer $n > L$ leaving remainder $r_1 \bmod m_1$ and $r_2 \bmod m_2$.” Checking is two modulo operations and an upward scan; producing requires the Chinese remainder theorem or a search. The high-asymmetry anchor.

WordCon (synthetic). “Write one sentence about X , exactly N words long, containing words w_1 and w_2 .” Checking is a token count and two substring tests. A high-asymmetry task in a natural-language rather than symbolic domain.

MBPP [20]. Python function synthesis with executable tests. The one family admitting a near-perfect external verifier, which lets us separate the ceiling imposed by coverage from the ceiling imposed by the selector.

GSM8K [13]. Grade-school word problems with a numeric answer. Checking means redoing the arithmetic, but the answer is a short canonical string, so voting applies.

ARC-Challenge [21]. Multiple-choice science questions. Verification is essentially re-answering; the answer space is tiny.

Logic (synthetic). Transitive-ordering puzzles: a shuffled chain of pairwise comparisons over 4–6 entities, asking for the extremum. Confirming a proposed answer means re-running the same derivation. The low-asymmetry anchor.

Three families are generated programmatically from a fixed seed. This is deliberate: synthetic families carry no benchmark-contamination risk, their difficulty is tunable, and their verifiers are exact rather than approximate. Each family is split 30/60 into a calibration and a held-out set *before any model is run*, since our central prediction claim requires that the split not be chosen after seeing results.

B. Models

We use the Qwen2.5-Instruct family [22] at 0.5B and 1.5B parameters, in `bfloat16` without quantisation, run locally with MLX on a single Apple M4 Pro. Using one family across scales isolates model size from architecture and tokenizer differences, and running locally makes the entire study reproducible at zero marginal cost—a deliberate choice given that our claims concern compute accounting. The released code runs a third scale (3B) unchanged; we report two because the full grid at three scales exceeds the compute we allotted, and we prefer two scales at full statistical power to three at half. Sampling uses temperature 0.7 and top- p 0.95; the judge and pairwise probes are run at temperature 0. Section VII-C

discusses what this does and does not license us to say about frontier-scale models.

C. Architectures

All selector-based architectures read the *same* pool of $n = 8$ independent samples per task, so differences between them are attributable to selection alone, with the generator held fixed and comparisons paired at the task level.

Single. One sample. The $k = 1$ reference point.

Sequential refine. One agent that generates, then revises its own answer for $r - 1$ further rounds, $r \in \{1, 2, 3, 4\}$. This is the compute-matched single-agent baseline and traces the frontier $a(\cdot)$.

Self-consistency. Majority vote over the canonical answers of k samples [5]. Undefined for families with no short canonical answer (WordCon, MBPP), which we report as such rather than forcing.

Ensemble + LLM judge. The same model, shown all k candidates in randomised order, picks one.

Ensemble + program verifier. A non-LLM checker derived from the problem statement alone selects the first passing candidate. Available for CSP, WordCon and MBPP. It consumes no model tokens.

Ensemble + oracle. Selects a correct candidate when one exists. Not deployable; reported as the coverage ceiling.

Debate. 3 agents over 2 or 3 rounds, each round conditioned on the others’ previous answers, with a final majority vote [4].

Selectors are evaluated at $k \in \{2, 4, 8\}$; for $k < 8$ we average over three random sub-pools to reduce variance.

a) *Preventing verifier leakage.*: The program verifier must not see anything the generator did not. For CSP and WordCon the constraints are fully stated in the prompt, so the checker is a direct transcription of the problem. For MBPP the prompt shows one test case, and the verifier may use *only* that test; grading uses the held-out remainder of the test list. Without this precaution the verifier would be scored on the same signal used to select, inflating its apparent efficiency.

D. Token accounting

Every generation records its own prompt and completion token counts, and an architecture’s cost is the sum over all calls it makes, including judge and debate turns. We count prompt tokens as well as completions because a judge call that reads eight candidates is expensive even when its output is one word. The program verifier’s cost is recorded as zero model tokens; it runs in milliseconds of CPU, which is the point of using it.

E. Statistics

All intervals are 95% percentile bootstrap intervals over *tasks* (10,000 resamples), since samples within a task are not independent. Architecture comparisons are paired on the same task set. For token-matched lift both arms are recomputed inside each bootstrap resample—the multi-agent accuracy and

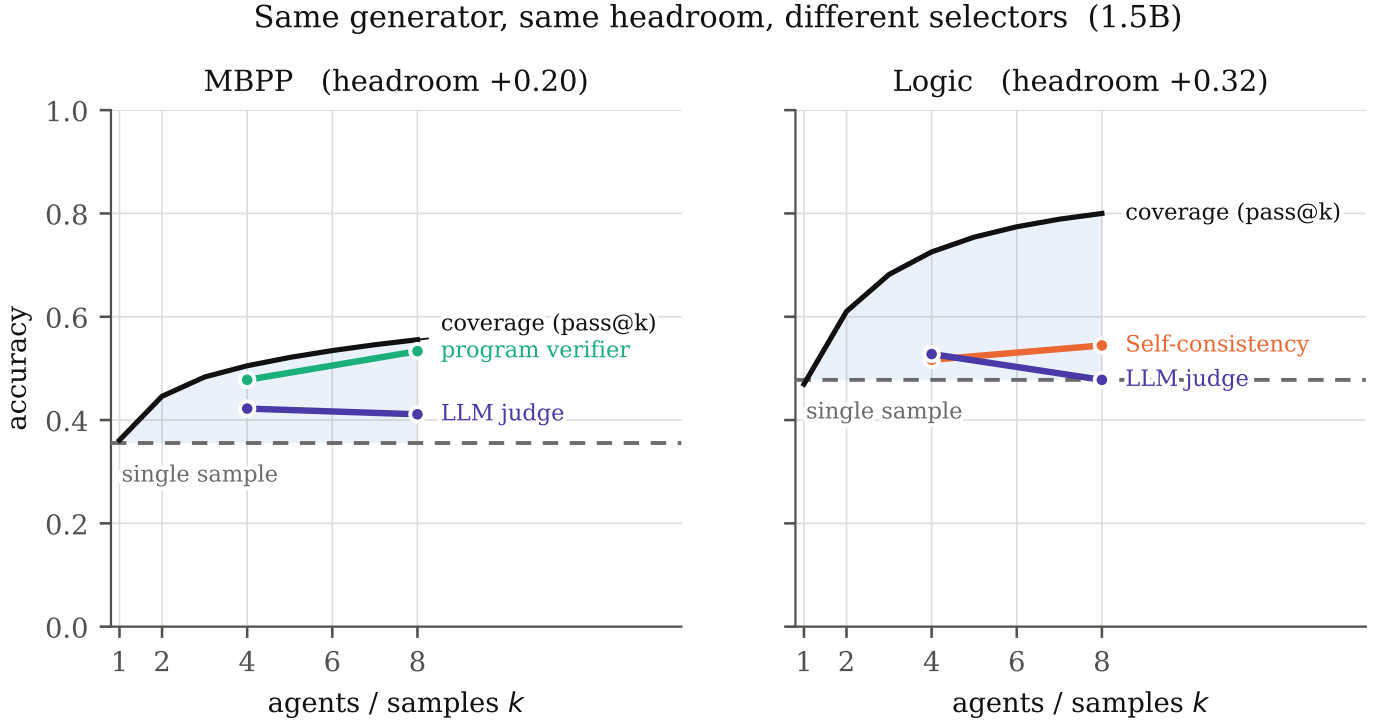


Fig. 1. The decomposition on a high- and a low-asymmetry family. The shaded band is coverage headroom—accuracy that already exists in the candidate pool. Where a selector’s curve sits inside that band is its selection efficiency η . The two panels share a generator and differ only in how checkable the task is.

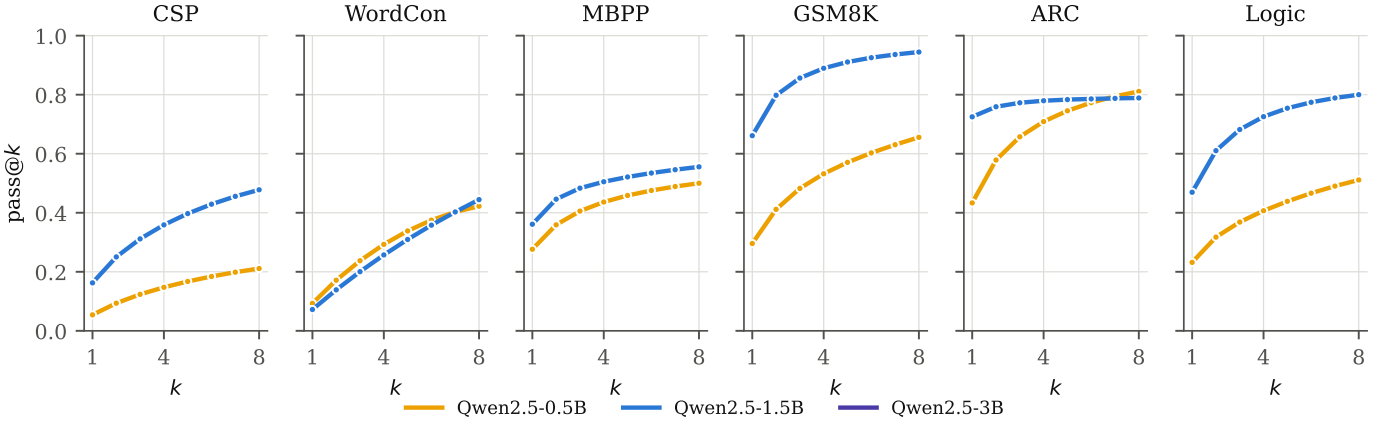


Fig. 2. Unbiased pass@ k by family and model. Headroom—the gap between $k = 1$ and $k = 8$ —is substantial almost everywhere, so the coverage factor is rarely what limits a multi-agent system.

the sequential frontier it is priced against—because the baseline is itself an estimate; holding it fixed would understate the interval. Where we report many comparisons at once, p -values are Holm–Bonferroni adjusted. Coverage uses the unbiased pass@ k estimator throughout.

V. RESULTS

We report five findings, in the order the framework implies: coverage exists (R1), selectors mostly fail to convert it (R2–R3), compute-matching removes most of what remains (R4), and the two cheap diagnostics predict all of this in advance (R5).

A. R1: Coverage headroom is large and nearly universal

Figure 2 shows pass@ k for every family and model. Consistent with the repeated-sampling literature [9, 10], coverage rises steeply with k : sampling eight times rather than once adds substantial headroom in nearly every cell. Table I gives the corresponding single-sample accuracies.

The practical reading is that the first factor of Equation (1) is usually *not* the binding constraint. Multi-agent systems fail, when they fail, on the second factor.

B. R2: The generator–verifier gap varies enormously, and is sometimes negative

Table II reports G and A per cell. The spread is the central empirical fact of this paper: the same model, asked the same

TABLE I. Accuracy by architecture and family, and mean token cost per task. The oracle row is the coverage ceiling: it is not deployable, and the distance between it and the deployable selectors is the accuracy the ensemble generated but could not identify.

Architecture	CSP	WordCon	MBPP	GSM8K	ARC	Logic
Single sample	0.21	0.10	0.36	0.64	0.70	0.48
Sequential refine	0.34	0.03	0.39	0.67	0.64	0.40
Self-consistency	0.23	—	—	0.87	0.71	0.54
Ensemble + LLM judge	0.20	0.07	0.41	0.69	0.70	0.48
Ensemble + program verifier	0.48	0.44	0.53	—	—	—
Debate (3×2)	0.19	0.02	0.38	0.53	0.72	0.53
<i>Ensemble + oracle (ceiling)</i>	0.48	0.44	0.56	0.94	0.79	0.80
<i>mean tokens per task</i>						
Single sample	366	78	125	309	101	81
Sequential refine	2,274	449	908	1,709	762	929
Ensemble + LLM judge	5,012	849	1,454	4,052	1,028	961
Debate (3×2)	2,532	637	1,093	2,194	736	662

TABLE II. The two cheap diagnostics measured on the 30-task calibration split, with judge position bias (the rate at which the first-listed candidate is chosen; 0.5 is unbiased) and order consistency (how often the judge names the same candidate when the two are presented in the opposite order; a judge that simply picks a position scores 0). G ranges from -0.27 to +0.42; 1 cells are negative, meaning the model prefers its own incorrect answers more often than its correct ones.

Model	Diagnostic	CSP	WordCon	MBPP	GSM8K	ARC	Logic
Qwen2.5-0.5B	G (verifier)	+0.00	+0.00	+0.00	+0.00	+0.00	+0.00
	A (plurality)	+0.03	—	—	+0.13	+0.03	-0.07
	position bias	1.00	0.00	0.20	1.00	1.00	1.00
	order consistency	0.00	0.00	0.20	0.00	0.00	0.00
Qwen2.5-1.5B	G (verifier)	+0.42	-0.27	+0.42	+0.05	+0.09	+0.05
	A (plurality)	+0.07	—	—	+0.30	+0.00	+0.00
	position bias	0.29	0.59	0.54	0.42	0.36	0.18
	order consistency	0.58	0.45	0.58	0.65	0.00	0.16

kind of question—“which of these two answers is right?”—ranges from near-perfect discrimination on families whose answers can be checked mechanically to *below chance* on families where checking means redoing the derivation. 1 of our cells have $G < 0$.

A negative gap deserves emphasis because it is not merely a small gain. It means a model critic is an *actively harmful* selector: it systematically prefers wrong answers, so adding it to a pipeline reduces accuracy below picking a candidate at random. Part of the mechanism is visible in the position bias column, where the smallest model chooses the first-listed candidate up to 1.00 of the time, reproducing at small scale the bias documented for stronger judges by Zheng et al. [16]. Order consistency falls as low as 0.00: on those cells the judge is not evaluating the candidates at all, it is naming a position, and reversing the presentation reverses its verdict.

Because a position-picking judge would otherwise inherit spurious discrimination from any imbalance in which side happens to hold the correct answer, every pair in our probe is presented twice, once in each order, and G averages over both. Under this counterbalancing a pure position-picker scores exactly chance by construction. We flag this because our own first implementation randomised the side independently per pair, which at $n \approx 20$ produced an apparent G of +0.40 on a cell whose judge chose the first option 100% of the time.

C. R3: Selection efficiency, not coverage, is what separates architectures

Figure 3 decomposes the architectures by η . Across 38 measured cells the median program verifier attains $\eta = 1.00$, the median self-consistency 0.19, and the median LLM judge 0.01, with a worst case of -0.39 ; 5 cells are negative.

Two caveats keep this honest. First, for CSP and WordCon the program verifier is a *complete* checker—it tests exactly the property the grader tests—so $\eta = 1$ there is true by construction and demonstrates only that the ceiling is attainable at zero token cost, not that verification is magically easy. The informative case is MBPP, where the verifier sees one test and grading uses held-out tests, so its efficiency is earned rather than assumed. Second, self-consistency requires a canonical short answer and is simply undefined for WordCon and MBPP; we report that rather than forcing a degenerate clustering.

D. R4: Token-matching removes most of the apparent benefit

This is the result with the most direct bearing on practice. Measured against a single sample—the comparison most commonly reported—the median architecture gains +0.036. Measured against a single agent given the same token budget, the median becomes -0.028, and 56% of all configurations are net negative. Of the 49 cells that appear to benefit under the naive comparison, 23 (47%) do not survive compute-matching.

The architectures do not fail equally, and the pattern is the one the framework predicts. Systems whose selector is an external program retain their advantage, because their

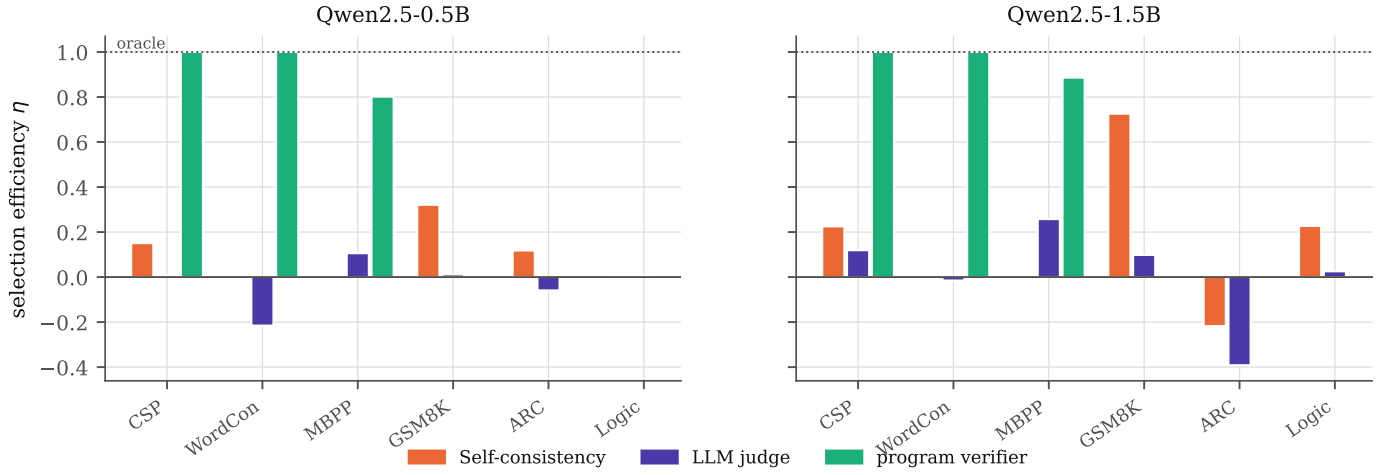


Fig. 3. Selection efficiency η at $k = 8$. A value of 1 means the selector captures all available headroom; 0 means it does no better than picking at random; negative means it does worse.

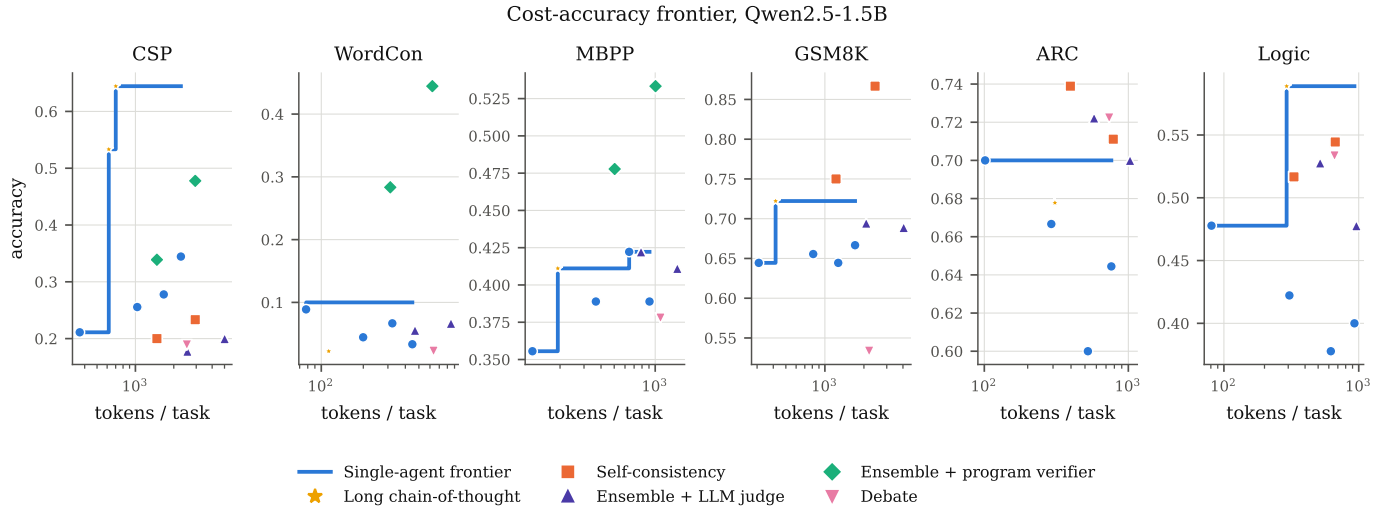


Fig. 4. Accuracy against token cost. The line is the single-agent frontier: the upper envelope of sequential refinement and long chain-of-thought at each budget. A multi-agent point above the line earns its compute; a point below it would have done better as one agent thinking longer.

TABLE III. Token-matched lift by family and architecture. * marks Holm–Bonferroni adjusted $p < 0.05$. Compare against the naive comparison against a single sample, which is what most of the literature reports.

Family	Self-consistency	LLM judge	program verifier	Debate (3×2)
CSP	−0.444*	−0.467*	−0.306*	−0.456*
WordCon	—	−0.044	+0.344*	−0.078
MBPP	—	−0.011	+0.111*	−0.044
GSM8K	+0.144*	−0.033	—	−0.189*
ARC	+0.039	+0.022	—	+0.022
Logic	−0.072	−0.111	—	−0.056

verification is both reliable and free. Systems whose selector is the model judging itself lose most or all of theirs, because they pay k times the tokens for a selector whose G does not support the choice. Debate is the most expensive arm and among the weakest per token, which is consistent with its gains being attributable to additional sampling rather than to the exchange of arguments—the interpretation Li et al. [6] reach from the opposite direction.

E. R5: The diagnostics predict lift before the system is built

Finally we test whether the framework is usable prospectively. Features come from the 30-task calibration split only; targets come from the 60-task held-out split; the split was fixed before any model ran.

A discriminating pair requires one correct and one incorrect candidate in the same pool, so on families where accuracy is very low or very high, G rests on few comparisons. We therefore pre-specified a minimum of 10 scored pairs for cells whose prediction depends on G , which excludes 2 cells; without that filter the same analysis gives $R^2 = 0.85$, so the

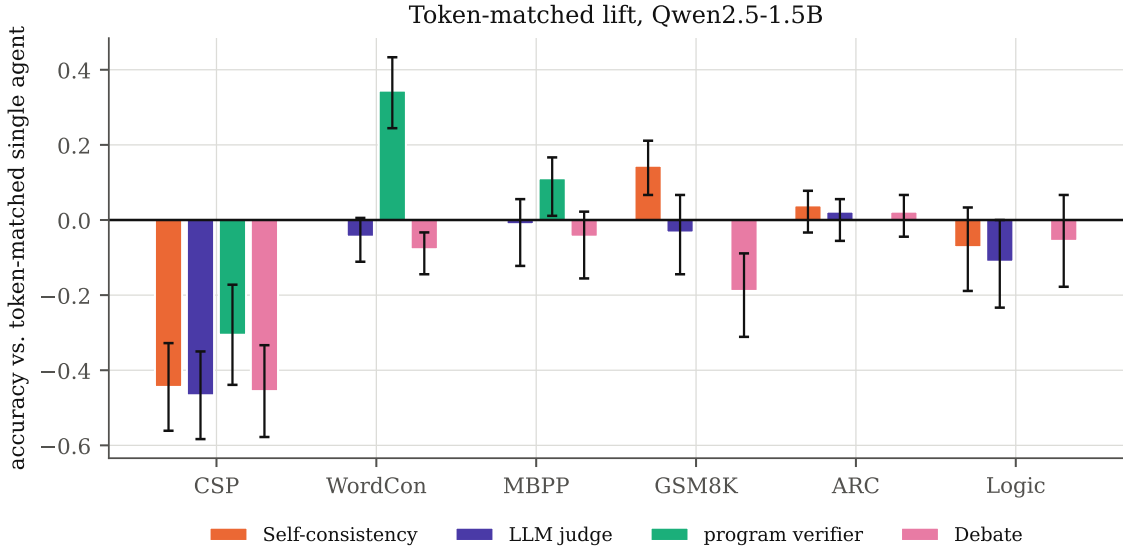


Fig. 5. Token-matched lift with 95% cluster-bootstrap intervals, in which both the multi-agent accuracy and the single-agent frontier are recomputed inside every resample.

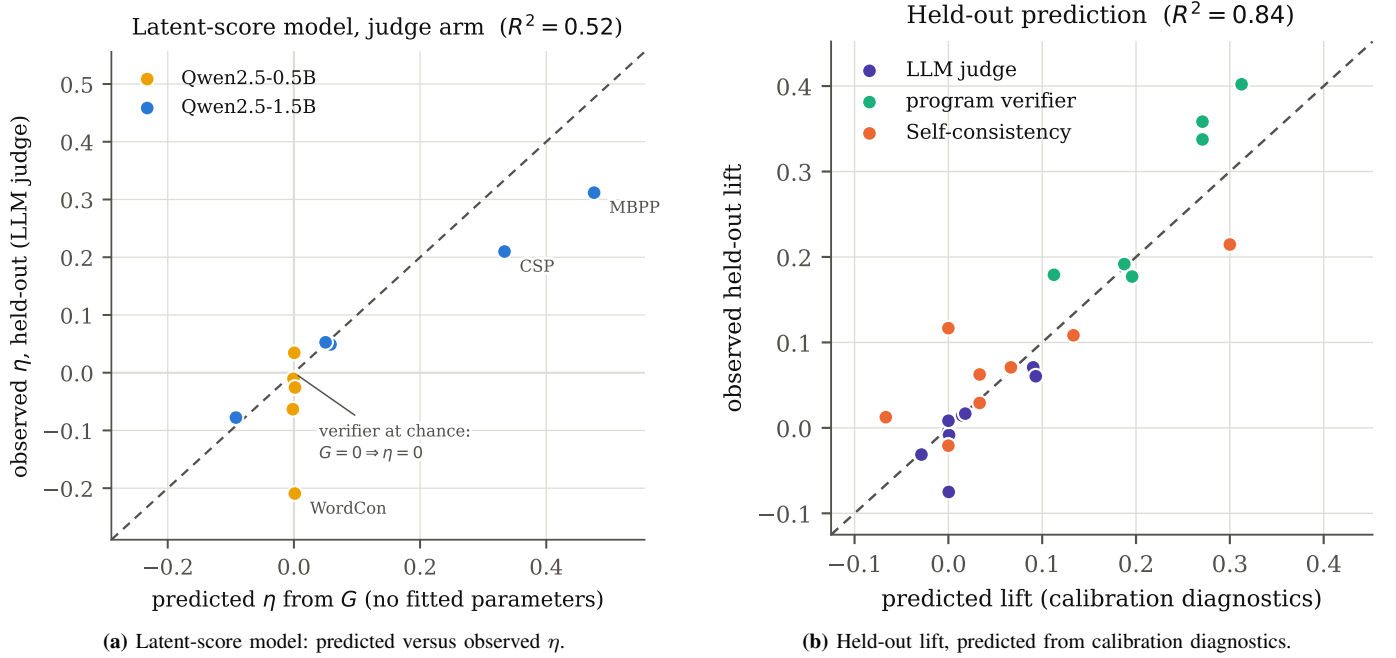


Fig. 6. Both panels compare against the identity line, not a fitted regression: the theory predicts the *value*, so a fitted slope would hide bias.

conclusion does not rest on the choice.

Across 24 cells, the a priori prediction— G and A measured on calibration, passed through Proposition 2, with no fitted parameters—attains $R^2 = 0.84$ against the identity line (Pearson $r = 0.92$, mean absolute error 0.036). Simply transferring measured η from calibration to held-out attains $R^2 = 0.80$, which upper-bounds what any diagnostic could achieve given sampling noise; the a priori prediction essentially matches it, so the cheap probe loses little against directly measuring the thing it estimates.

Restricted to the judge arm alone—where the latent-score model does all of the work, rather than the coarse separation between verifier types—the prediction attains $R^2 = 0.52$ over 10 cells (Figure 6a). Two features of that panel are worth

stating plainly. Every cell with $G = 0$ is predicted to have $\eta = 0$ exactly, which is not a fitted success but a property of the model; the observed values scatter around zero as sampling noise. And at the high end the model *over*-predicts: the two cells with the largest gaps sit visibly below the identity line. We take this as the expected cost of assuming conditionally independent, equally dispersed scores—a listwise judge that sees all candidates at once violates both—and we report it rather than introducing a free parameter to absorb it.

Turned into the binary decision of Equation (7)—build the ensemble, or spend the tokens on one agent—the rule agrees with the token-matched outcome in 73% of 26 cells, at precision 0.64 and recall 0.70. We report that number against the right yardstick rather than against chance: because multi-

Both diagnostics are model-dependent, not task-intrinsic

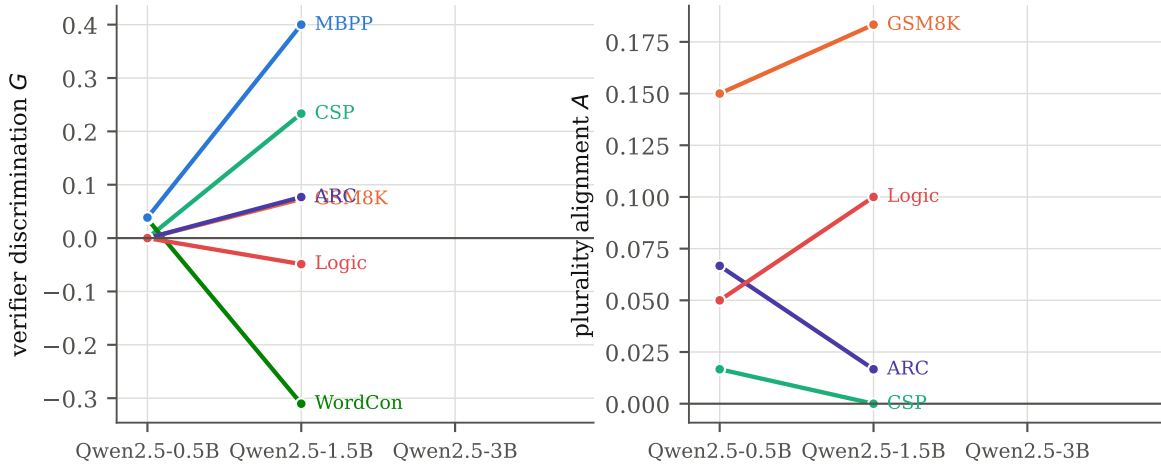


Fig. 7. Both diagnostics across model scale. They are properties of a (model, task) pair, not of the task alone, so they must be re-measured when either changes.

agent architectures fail so often in our data, simply answering “do not build” every time is already correct 61.5% of the time, so the diagnostic buys 11.5 percentage points over that baseline. This is a real but modest gain, and it is the weakest of our results. The continuous prediction is the claim we would stand behind: the diagnostics estimate *how much* an architecture will gain far better than they resolve the knife-edge cases where the gain is near zero, which is precisely where a binary rule is forced to commit. A practitioner is better served by the predicted magnitude and its uncertainty than by the rule’s verdict.

That the majority baseline sits at 61.5% is itself the finding worth carrying away: across 26 architecture–task pairs drawn from the designs the literature recommends, the prior that multi-agent structure will not survive compute-matching is correct more often than not.

F. R6: How the diagnostics move with scale

Figure 7 contrasts G and A at 0.5B and 1.5B. The important qualitative point is that neither is a property of the task alone: the same family can move from a negative to a positive gap as the model grows, which means a multi-agent design that fails at one scale may succeed at another and vice versa. This is the mechanism we think underlies much of the disagreement in the literature. We state explicitly what two points cannot do: they show the diagnostics are model-dependent, and they establish no trend and license no extrapolation. A third scale point is the single most valuable addition to this study, and the released code runs it unchanged.

VI. CASE STUDY: CITATION VERIFICATION

The six families of Section IV are controlled by construction. To check that the framework says something useful about a task people actually deploy language models on, we apply it to reference generation—a setting where fabricated output is a known and consequential failure mode, and where an external

registry supplies a near-perfect verifier for the price of an HTTP request.

The task “produce a real published reference on topic X ” maps onto the framework exactly. A generator proposes k candidate references; a selector picks one; ground truth is whether the reference exists. We ran this with Qwen2.5-1.5B over 40 research topics spanning the sciences, drawing $k = 8$ candidate references per topic, and resolved every candidate against the OpenAlex catalogue by fuzzy title match (threshold 0.88), which supplies both ground truth and one of the two selectors under comparison.

a) *Resolution failures are not fabrications.*: A registry lookup can fail because the paper does not exist or because the service is temporarily unavailable, and conflating the two would convert a network condition into a measured hallucination. We retry with exponential backoff and record any candidate we still cannot resolve as *unresolved* rather than absent; 0 candidates were excluded on this basis, and a topic contributes to the analysis only if all eight of its candidates resolved.

A. Results

The generator fabricates at scale. Of 40×8 candidates, 36 were not parseable as references at all; of the 284 that were, 97.2% could not be matched to any indexed work. A single candidate is therefore real with probability 0.03, and at least one of eight resolves for only 0.15 of topics.

a) *The rate depends on the match threshold, the conclusion does not.*: Title matching is a similarity judgement, so we report the whole curve rather than one cutoff. At our threshold of 0.88 the fabrication rate is 97.2%; relaxing it to 0.80 gives 87% and to 0.70 gives 69%. The median match ratio across all candidates is 0.63, far below any defensible cutoff. We keep 0.88 because inspection shows looser thresholds admit genuinely different papers—a fabricated “Graph Neural Networks for Molecular Property Prediction: A Systematic Review” matches a real but distinct paper on regularised graph

networks at 0.71—so the looser numbers understate fabrication rather than correcting it. What survives every threshold is the qualitative claim: most of what this generator proposes does not exist.

b) *Neither verifier faces the same problem.:*

The model as its own critic. Asked whether each reference is real, the model is not credulous but indiscriminately sceptical: it answers “real” for only 15% of candidates, rejecting 0.85 of fabrications while also rejecting three quarters of the genuine ones (TPR = 0.25). Its balanced accuracy is barely above chance, and its pairwise gap is $G = -0.67$ —below chance, meaning that shown one real and one fabricated reference it prefers the fabrication more often than not. Its realised selection efficiency is $\eta = 0.00$ at a cost of 32,206 tokens.

The external registry. The same selection made by a registry lookup attains $\eta = 1.00$ at zero model tokens, raising accuracy from 0.03 to 0.15.

Given only the measured G , the latent-score model of Proposition 2 predicts $\hat{\eta} = -0.18$ —correctly anticipating that this critic is worse than useless, without the selection being run.

c) *What this case study cannot support.:* Coverage here is very low: only six of 40 topics contain any real reference among their eight candidates, so the comparison between selectors rests on a handful of items and the efficiencies should be read as directional rather than precise. The fabrication rate and the gap G rest on far more data (284 candidates and every discriminating pair available) and we report those with more confidence. We also note that a 1.5B model is a worst case for this task; we would expect a frontier model to fabricate far less, and we make no claim about where its G would sit.

B. What the case study shows

Two agents that look alike in an architecture diagram—one checks citations, one polishes prose—are not alike at all under this framework. The first has an external oracle available for the price of an HTTP request and converts nearly all of its coverage headroom. The second has no oracle, and a model asked to supply one performs near chance. The generalisable lesson is that the question worth asking about a proposed agent is not how capable it is but whether anything cheap can tell when it is wrong. Where an external check exists, use it: in our measurements it dominated the model-critic alternative on both accuracy and cost, which is not a trade-off but a strict improvement.

VII. DISCUSSION

A. What the framework says to build

Read as engineering advice, the decomposition gives a short checklist that can be run in an afternoon on 30 labelled tasks, before any orchestration code is written.

- 1) **Measure the headroom.** Sample k answers per task and compute $C(k) - p_1$. If it is small, no architecture will help, and the effort belongs in a better generator or a better prompt rather than in orchestration.

- 2) **Measure the gap G .** Show the model one correct and one incorrect candidate and ask which is right. If $G \approx 0$, a language-model critic will not convert the headroom, no matter how the critique is prompted or how many rounds it runs.
- 3) **Look for an external check first.** A program that validates the task’s own stated constraints is the highest-efficiency selector available and, unlike a judge, consumes no model tokens. Much of the benefit attributed to multi-agent design in practice is really the benefit of having a checker.
- 4) **Price the alternative.** Compare the predicted gain against what one agent achieves with the same budget spent serially. This is the comparison that decides whether the architecture is worth its complexity.

The uncomfortable implication for multi-agent design is that the most common pattern—adding a language-model critic or reviewer agent—is the one our framework is least optimistic about, because it depends entirely on G being large for the specific model and task at hand, and G is exactly what practitioners do not currently measure.

B. Reconciling conflicting reports

The framework explains why the literature disagrees without anyone being wrong. Studies that report large multi-agent gains tend to use tasks with substantial coverage headroom and either a strong external verifier or an answer distribution peaked on the truth; studies that report little benefit tend to use tasks where verification is as hard as generation, or compare against stronger baselines. Because G and A vary across both tasks and models, two papers can run the same architecture on different benchmarks and reach opposite conclusions that are both correct. The prescription is not to settle the argument in general but to measure the two diagnostics in each new setting.

C. Limitations

a) *Model scale.:* Our experiments use open models at 0.5B and 1.5B parameters, run locally. This was a deliberate choice—it makes the study exactly reproducible at zero marginal cost, which matters for a paper whose central claim is about compute accounting—but it is also the main limit on what we can claim. Frontier models are better verifiers, so G should be larger for them and selection efficiency correspondingly higher. Our scale analysis (Section V) contrasts the two scales we have, which shows the diagnostics are model-dependent rather than task-intrinsic but establishes no trend: two points determine a line and test nothing. Extrapolating to models two orders of magnitude larger is not something our data licenses, and we make no such claim. What we do claim is scale-independent: the *decomposition* is an identity, the need for compute-matched baselines is a matter of accounting, and the diagnostics remain measurable at any scale. Whether the specific numbers transfer is an empirical question we cannot answer here.

b) *The latent-score model is a simplification.:* Definition 3 assumes verifier scores are conditionally independent

across candidates and equally dispersed for correct and incorrect ones. Real judges violate both: they see all candidates at once, and a confident wrong answer may attract a higher score than a hedged right one. We report where the prediction holds and where it breaks rather than claiming the model is correct.

c) Task families are a sample, not a census.: Six families cannot span every kind of verification asymmetry. Our synthetic families are exactly checkable by construction, which is what makes them useful anchors, but it also makes them unrepresentative of open-ended work where no ground truth exists at all—precisely the regime where much multi-agent deployment happens, and where our framework predicts the least benefit but can offer no measurement.

d) Architectures are a sample too.: We test the designs that dominate the literature—voting, judging, debate, refinement—but not tool-using agents, long-horizon planning, or role-specialised pipelines where agents solve genuinely different sub-problems. Our framework applies to the selection step of any such system; it does not model the decomposition step.

e) Single-answer tasks.: The decomposition assumes a task has a selectable answer. It does not directly cover systems whose agents each contribute a different part of one artifact, which is how many deployed pipelines—including the one in our case study—actually work. We address that gap only structurally, by classifying which stages admit an external check.

VIII. CONCLUSION

We set out to answer a question practitioners face constantly and currently answer by guesswork: is a multi-agent architecture worth building for this task? The answer we arrive at is that the question is well-posed and cheaply decidable. An ensemble’s accuracy is its coverage times its selection efficiency, selection efficiency is predictable from a pairwise probe that costs a few dozen model calls, and the honest baseline is one agent given the same tokens. Measured this way, multi-agent structure stops looking like a general technique and starts looking like what it is: an effective way to exploit an asymmetry between generating and checking, and an expensive way to spend tokens when that asymmetry is absent.

APPENDIX A REPRODUCIBILITY

Every result in this paper was produced by the code released alongside it, on a single Apple M4 Pro (24 GB unified memory) with no API access and no paid compute. The pipeline is:

- 1) `gvgap/build_data.py` — builds the frozen 540-task suite. Synthetic families are generated from seed 20260919; benchmark families are subsampled with the same seed. The calibration/held-out split is assigned here, before any model runs.
- 2) `gvgap/run.py` — runs all architectures for all models. Every generation is cached by a hash of (model, prompt,

max tokens, temperature, seed), so the experiment is re-summable and re-running it is free.

- 3) `gvgap/analysis.py`, `predict.py`, `report.py`, `figures.py` — aggregation, hypothesis tests, tables and figures.

Total generations: 21,847. Models are Qwen2.5-Instruct at 0.5B and 1.5B [22] in `bfloat16` via `MLX`; `gvgap/run.py` runs a 3B scale point unchanged by uncommenting one line.

a) Estimator validation.: The estimators the paper’s claims rest on are tested against ground truth rather than against themselves (`gvgap/test_metrics.py`): `pass@k` is checked against exhaustive enumeration of all k -subsets for every $n \leq 8$; the bootstrap interval is checked for empirical coverage (0.963 against a nominal 0.95); the paired bootstrap is checked for its false-positive rate under the null (0.057 at $\alpha = 0.05$); and the latent-score model is checked against its own boundary conditions, including that it reduces exactly to uniform selection (c/k) when $G = 0$ and is monotone in G .

b) Bibliography hygiene.: Every reference in this paper was resolved against the arXiv API before inclusion, and entries are generated from the returned metadata rather than transcribed. This caught four incorrect titles in our own first draft of the reference list — an error rate of roughly one in six — which is a small illustration of the case study’s point in Section VI: a cheap external check finds errors that careful reading does not.

APPENDIX B TASK FAMILY DETAILS

a) CSP.: Moduli are drawn from $\{3, \dots, 9\}$ with $k = 2$ congruences and lower bounds in $\{10, 20, 30, 50, 100\}$. The answer is verified structurally: we re-check the congruences and confirm minimality by an upward scan, rather than trusting the stored answer.

b) Logic.: 4–6 entities are placed in a total order; the $n - 1$ adjacent comparisons are emitted in random surface form (“ X is taller than Y ” or “ Y is shorter than X ”) and shuffled. The question asks for the maximum or minimum.

c) WordCon.: Target lengths are 6–9 words with two required content words drawn from a fixed topic list. Grading tokenises on `[A-Za-z’]` and requires an exact length match plus both required words as case-insensitive substrings.

d) MBPP.: We use the full test split, shuffled with the suite seed. Candidates are executed in a subprocess with an 8-second timeout. The selector may use only the single test shown in the prompt; grading uses the remaining tests.

APPENDIX C PROMPTS

All architectures share one system prompt per family, so that differences are attributable to orchestration rather than prompt engineering. The judge is shown candidates in an order permuted per task with a fixed seed, and the permutation is recorded so that position bias can be measured separately (reported in Table II). The pairwise probe presents exactly one correct and one incorrect candidate in randomised order. Full prompt text is in `gvgap/prompts.py`.

APPENDIX D FIGURE DESIGN

Categorical colours are assigned in a fixed order so that an architecture carries the same hue in every figure, and the palette was checked programmatically for colour-vision-deficiency separation (worst adjacent pair $\Delta E = 9.1$ in OKLab $\times 100$, above the threshold of 8) rather than by eye. Three hues fall below a 3:1 contrast ratio against the page, so every figure also carries axis labels and a corresponding table, and identity is never carried by colour alone.

REFERENCES

- [1] Q. Wu *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [2] S. Hong *et al.*, “Metagpt: Meta programming for a multi-agent collaborative framework,” in *ICLR*, 2024, arXiv:2308.00352.
- [3] C. Qian *et al.*, “Chatdev: Communicative agents for software development,” in *ACL*, 2024, arXiv:2307.07924.
- [4] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving factuality and reasoning in language models through multiagent debate,” in *ICML*, 2024, arXiv:2305.14325.
- [5] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, and S. Narang, “Self-consistency improves chain of thought reasoning in language models,” in *ICLR*, 2023, arXiv:2203.11171.
- [6] J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye, “More agents is all you need,” *arXiv preprint arXiv:2402.05120*, 2024.
- [7] M. Cemri *et al.*, “Why do multi-agent llm systems fail?” *arXiv preprint arXiv:2503.13657*, 2025.
- [8] S. Kapoor, B. Stroebel, Z. S. Siegel, N. Nadgir, and A. Narayanan, “Ai agents that matter,” *arXiv preprint arXiv:2407.01502*, 2024.
- [9] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, and C. Ré, “Large language monkeys: Scaling inference compute with repeated sampling,” *arXiv preprint arXiv:2407.21787*, 2024.
- [10] C. Snell, J. Lee, K. Xu, and A. Kumar, “Scaling llm test-time compute optimally can be more effective than scaling model parameters,” *arXiv preprint arXiv:2408.03314*, 2024.
- [11] B. Stroebel, S. Kapoor, and A. Narayanan, “The limits of inference scaling through resampling,” *arXiv preprint arXiv:2411.17501*, 2024.
- [12] M. Chen *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [13] K. Cobbe *et al.*, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [14] H. Lightman *et al.*, “Let’s verify step by step,” in *ICLR*, 2024, arXiv:2305.20050.
- [15] Y. Li, Z. Lin, S. Zhang, Q. Fu, B. Chen, J.-G. Lou, and W. Chen, “Making large language models better reasoners with step-aware verifier,” *arXiv preprint arXiv:2206.02336*, 2022.
- [16] L. Zheng *et al.*, “Judging llm-as-a-judge with mt-bench and chatbot arena,” in *NeurIPS Datasets and Benchmarks*, 2023, arXiv:2306.05685.
- [17] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, and X. Song, “Large language models cannot self-correct reasoning yet,” in *ICLR*, 2024, arXiv:2310.01798.
- [18] K. Valmeekam, M. Marquez, and S. Kambhampati, “Can large language models really improve by self-critiquing their own plans?” *arXiv preprint arXiv:2310.08118*, 2023.
- [19] A. Madaan *et al.*, “Self-refine: Iterative refinement with self-feedback,” in *NeurIPS*, 2023, arXiv:2303.17651.
- [20] J. Austin *et al.*, “Program synthesis with large language models,” *arXiv preprint arXiv:2108.07732*, 2021.
- [21] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, and C. Schoenick, “Think you have solved question answering? try arc, the ai2 reasoning challenge,” *arXiv preprint arXiv:1803.05457*, 2018.
- [22] Qwen *et al.*, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.